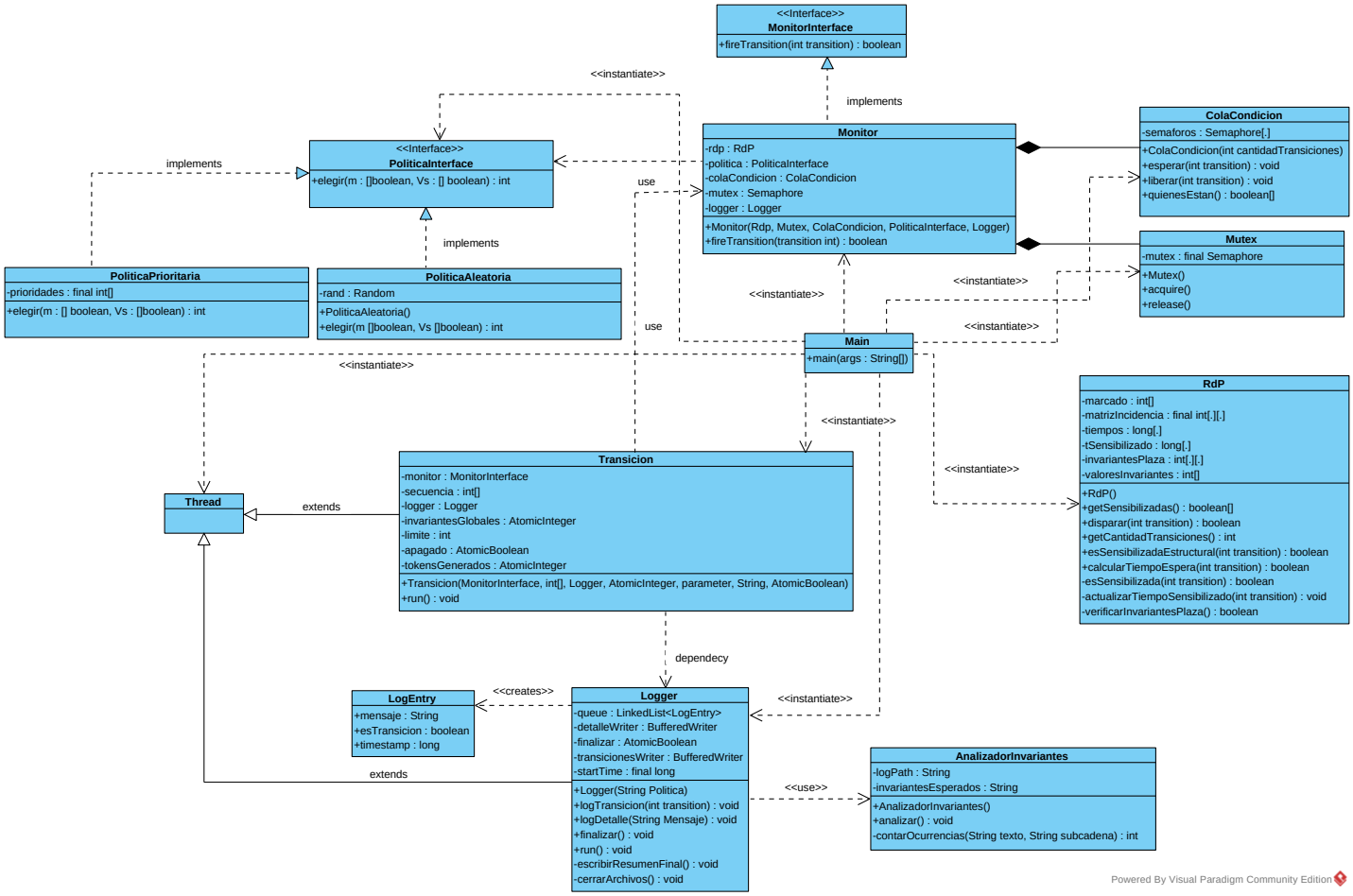
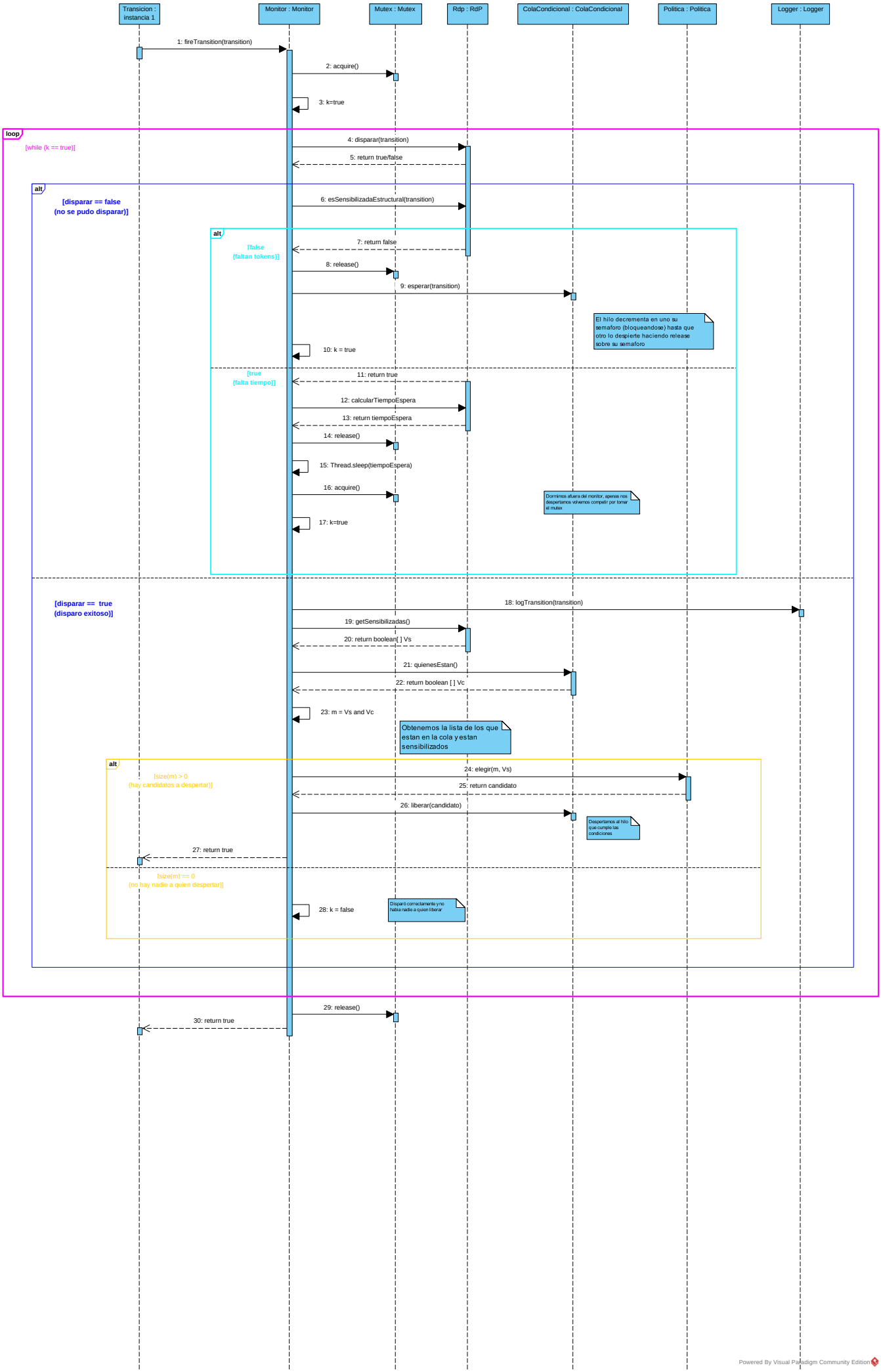

<<Interface>>
MonitorInterface
+fireTransition(int transition) : boolean
implements
<<instantiate>>
Monitor
-rdp : RdP
-politica : PoliticaInterface
-colaCondicion : ColaCondicion
-mutex : Semaphore
-logger : Logger
+Monitor(Rdp, Mutex, ColaCondicion, PoliticaInterface, Logger)
+fireTransition(transition int) : boolean
ColaCondicion
-semaforos : Semaphore[.]
+ColaCondicion(int cantidadTransiciones)
+esperar(int transition) : void
+liberar(int transition) : void
+quienesEstan() : boolean[]
<<Interface>>
PoliticaInterface
+elegir(m : []boolean, Vs : [] boolean) : int
implements
use
Mutex
-mutex : final Semaphore
+Mutex()
+acquire()
+release()
PoliticaPrioritaria
-prioridades : final int[]
+elegir(m : [] boolean, Vs : []boolean) : int
PoliticaAleatoria
-rand : Random
+PoliticaAleatoria()
+elegir(m []boolean, Vs []boolean) : int
<<instantiate>>
Main
+main(args : String[])
RdP
-marcado : int[]
-matrizIncidencia : final int[.][.]
-tiempos : long[.]
-tSensibilizado : long[.]
-invariantesPlaza : int[.][.]
-valoresInvariantes : int[]
+RdP()
+getSensibilizadas() : boolean[]
+disparar(int transition) : boolean
+getCantidadTransiciones() : int
+esSensibilizadaEstructural(int transition) : boolean
+calcularTiempoEspera(int transition) : boolean
-esSensibilizada(int transition) : boolean
-actualizarTiempoSensibilizado(int transition) : void
-verificarInvariantesPlaza() : boolean
Thread
extends
Transicion
-monitor : MonitorInterface
-secuencia : int[]
-logger : Logger
-invariantesGlobales : AtomicInteger
-limite : int
-apagado : AtomicBoolean
-tokensGenerados : AtomicInteger
+Transicion(MonitorInterface, int[], Logger, AtomicInteger, parameter, String, AtomicBoolean)
+run() : void
dependecy
LogEntry
+mensaje : String
+esTransicion : boolean
+timestamp : long
<<creates>>
Logger
-queue : LinkedList<LogEntry>
-detalleWriter : BufferedWriter
-finalizar : AtomicBoolean
-transicionesWriter : BufferedWriter
-startTime : final long
+Logger(String Politica)
+logTransicion(int transition) : void
+logDetalle(String Mensaje) : void
+finalizar() : void
+run() : void
-escribirResumenFinal() : void
-cerrarArchivos() : void
<<use>>
AnalizadorInvariantes
-logPath : String
-invariantesEsperados : String
+AnalizadorInvariantes()
+analizar() : void
-contarOcurrencias(String texto, String subcadena) : int
Powered By Visual Paradigm Community Edition


Transicion : instancia 1
Monitor : Monitor
Mutex : Mutex
Rdp : RdP
ColaCondicional : ColaCondicional
Politica : Politica
Logger : Logger
1: fireTransition(transition)
2: acquire()
3: k=true
loop
[while (k == true)]
4: disparar(transition)
5: return true/false
alt
[disparar == false
(no se pudo disparar)]
6: esSensibilizadaEstructural(transition)
alt
[false
(faltan tokens)]
7: return false
8: release()
9: esperar(transition)
El hilo decrementa en uno su semaforo (bloqueandose) hasta que otro lo despierte haciendo release sobre su semaforo
10: k = true
[true
(falta tiempo)]
11: return true
12: calcularTiempoEspera
13: return tiempoEspera
14: release()
15: Thread.sleep(tiempoEspera)
16: acquire()
Dormimos afuera del monitor, apenas nos despertamos volvemos compatir por tomar el mutex
17: k=true
[disparar == true
(disparo exitoso)]
18: logTransition(transition)
19: getSensibilizadas()
20: return boolean[ ] Vs
21: quienesEstan()
22: return boolean [ ] Vc
23: m = Vs and Vc
Obtenemos la lista de los que estan en la cola y estan sensibilizados
alt
[size(m) > 0
(hay candidatos a despertar)]
24: elegir(m, Vs)
25: return candidato
26: liberar(candidato)
Despertamos al hilo que cumple las condiciones
27: return true
[size(m) == 0
(no hay nadie a quien despertar)]
28: k = false
Disparó correctamente y no habia nadie a quien liberar
29: release()
30: return true
Powered By Visual Paradigm Community Edition